# An Intro to Video4Linux2

Presented by
Kowshic Shankar R

# What is Video4Linux

- Video4Linux (V4L for short) is a collection of device drivers and an API for supporting realtime video capture on Linux systems.
- It supports USB webcams, TV tuners, CSI cameras, and related devices, standardizing their output, so programmers can easily add video support to their applications.

# What is ioctl()

- The ioctl() system call manipulates the underlying device parameters of special files.
- The first argument fd must be an open file descriptor.
- The second argument is a device-dependent operation code.
- The third argument is an untyped pointer to memory.

# ioctl()

## Syntax:

```
#include <sys/ioctl.h>

int ioctl(int fd, unsigned long op, ...);
```

## Return Value:

Usually, on success zero is returned. A few ioctl() operations use the return value as an output parameter and return a nonnegative value on success.

ERRORS

- EBADF fd is not a valid file descriptor.
- EFAULT argp references an inaccessible memory area.
- EINVAL op or argp is not valid.
- ENOTTY fd is not associated with a character special device.
- ENOTTY The specified operation does not apply to the kind of object that the file descriptor fd references.

# Opening a Video Device

- Unlike opening a normal file in c using fopen(), here we use a open() function from fcntl.h library.
- Return non negative number denoting the index of file descriptor on the table.

```
#include <fcntl.h>
    fd = open("/dev/video0", O_RDWR);
    if (fd == -1) {
        perror("open");
        return EXIT_FAILURE;
        }
```

# Querying Device Capabilities

We use the ioctl command , with op code VIDIOC_QUERYCAP and a pointer to a structure v4l2_capability defined in linux/videodev2.h

```
#include <sys/ioctl.h>
#include <linux/videodev2.h>
    struct v4l2_capability cap;
    memset(&cap, 0, sizeof(cap));
    if (ioctl(fd, VIDIOC_QUERYCAP, &cap) == -1) {
        perror("VIDIOC_QUERYCAP");
        close(fd);
        return EXIT_FAILURE;
    }

    printf("Driver : %s\n", cap.driver);
    printf("Card   : %s\n", cap.card);
```

# ioctl VIDIOC_QUERYCAP

- All V4L2 devices support the VIDIOC_QUERYCAP ioctl.
- It is used to identify kernel devices compatible with this specification and to obtain information about driver and hardware capabilities.
- The ioctl takes a pointer to a struct v4l2_capability which is filled by the driver.
- When the driver is not compatible with this specification the ioctl returns an EINVAL error code.

# struct v4l2_capability

| Field | Type | Description |
|---|---|---|
| driver | __u8[16] | Name of the driver. |
| card | __u8[32] | Name of the device. |
| bus_info | __u8[32] | Location of the device in the system. |
| version | __u32 | Version number of the driver. |
| capabilities | __u32 | Available capabilities of the physical device. |
| device_caps | __u32 | Capabilities of the opened device node. |
| reserved[3] | __u32[3] | Reserved for future extensions. |

# Device Capability Flags

| Flag | Description |
| --- | --- |
| V4L2_CAP_VIDEO_CAPTURE | Supports the single-planar Video Capture API. |
| V4L2_CAP_VIDEO_CAPTURE_MPLANE | Supports the multi-planar Video Capture API. |
| V4L2_CAP_STREAMING | Supports the streaming I/O method. |
| V4L2_CAP_DEVICE_CAPS | Driver fills the device_caps field. |

# Setting Video Format

- We use the ioctl command , with op code VIDIOC_S_FMT and a pointer to a structure v4l2_format defined in linux/videodev2.h.
- The v4l2_format structure should be passed with necessary elements initialized like type, width, height, pixelformat and field.
- We can use VIDIOC_G_FMT similarly to get the current v4l2_format.

```
#include <sys/ioctl.h>
#include <linux/videodev2.h>
    struct v4l2_format fmt;
    memset(&fmt, 0, sizeof(fmt));
    fmt.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    fmt.fmt.pix.width = WIDTH;
    fmt.fmt.pix.height = HEIGHT;
    fmt.fmt.pix.pixelformat = V4L2_PIX_FMT_YUYV;
    fmt.fmt.pix.field = V4L2_FIELD_ANY;
    if (ioctl(fd, VIDIOC_S_FMT, &fmt) == -1) {
        perror("VIDIOC_S_FMT");
        close(fd);
        return EXIT_FAILURE;
    }
```

# ioctl VIDIOC_S_FMT

- When the application calls the VIDIOC_S_FMT ioctl with a pointer to a struct v4l2_format structure the driver checks and adjusts the parameters against hardware abilities.
- Drivers should not return an error code unless the type field is invalid.
- On success the driver may program the hardware, allocate resources and generally prepare for data exchange.
- Finally the VIDIOC_S_FMT ioctl returns the current format parameters as VIDIOC_G_FMT does.
- Very simple, inflexible devices may even ignore all input and always return the default parameters.

# struct v4l2_format

| Field | Type | Description |
|---|---|---|
| type | __u32 | Type of the data stream. |
| fmt.pix | v4l2_pix_format | Single-planar video capture or output format. |
| fmt.pix_mp | v4l2_pix_format_mplane | Multi-planar video capture or output format. |
| fmt.win | v4l2_window | Parameters for an overlaid image. |
| fmt.vbi | v4l2_vbi_format | Raw VBI capture or output parameters. |
| fmt.sliced | v4l2_sliced_vbi_format | Sliced VBI capture or output parameters. |
| fmt.sdr | v4l2_sdr_format | SDR capture or output format parameters. |

# struct v4l2_pix_format

| Field | Type | Description |
|---|---|---|
| width | __u32 | Image width in pixels. |
| height | __u32 | Image width in pixels. |
| pixelformat | __u32 | Pixel format or type of compression. |
| field | enum v4l2_field | Video field order. |
| bytesperline | __u32 | Distance in bytes between adjacent image lines. |
| sizeimage | __u32 | Size in bytes required for a complete image. |
| colorspace | enum v4l2_colorspace | Color space of the image. |
| priv | __u32 | Indicates whether extended fields are valid. |
| flags | __u32 | Format-specific flags. |

With ioctl VIDIOC_ENUM_FMT we can query the available pixel formats.

# Requesting MMAP Buffers

```
#include <sys/ioctl.h>
#include <linux/videodev2.h>
#define NUM_BUFFERS 4
    struct v4l2_requestbuffers req;
    memset(&req, 0, sizeof(req));
    req.count = NUM_BUFFERS;
    req.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
    req.memory = V4L2_MEMORY_MMAP;
    if (ioctl(fd, VIDIOC_REQBUFS, &req) == -1) {
        perror("VIDIOC_REQBUFS");
        close(fd);
        return EXIT_FAILURE;
    }
    printf("Number of buffers: %u\n", req.count);
```

- Buffers are memory locations that store the data from the device.
- We need to request Buffers for MMAP.
- We use ioctl VIDIOC_REQBUFS with argument v4l2_requestbuffers that contain count of buffer needed and type.

# ioctl VIDIOC_REQBUFS

- This ioctl is used to initiate memory mapped, user pointer or DMABUF based I/O.
- Memory mapped buffers are located in device memory and must be allocated with this ioctl before they can be mapped into the application’s address space.
- To allocate device buffers applications initialize all fields of the struct v4l2_requestbuffers structure.
- They set the type field to the respective stream or buffer type, the count field to the desired number of buffers, memory must be set to the requested I/O method and the reserved array must be zeroed.

# struct v4l2_requestbuffers

| Field | Type | Description |
|---|---|---|
| count | __u32 | The number of buffers requested or granted. |
| type | __u32 | Type of the stream or buffers. |
| memory | __u32 | Requested I/O method: MMAP, DMABUF or USERPTR. |
| reserved[2] | __u32 | Reserved for future extensions and must be zeroed. |

# mmap()

- mmap() creates a new mapping in the virtual address space of the calling process.
- The starting address for the new mapping is specified in addr.
- The length argument specifies the length of the mapping (which must be greater than 0).
- If addr is NULL, then the kernel chooses the (page-aligned) address at which to create the mapping, this is the most portable method of creating a new mapping.
- On success, mmap() returns a pointer to the mapped area.

# munmap()

- The munmap() system call deletes the mappings for the specified address range, and causes further references to addresses within the range to generate invalid memory references.
- The region is also automatically unmapped when the process is terminated.
- On success, munmap() returns 0.

# mmap() & munmap()

Syntax:

```
#include <sys/mman.h>
void *mmap(
        void addr[length], size_t length;
        int prot, int flags, int fd, off_t offset);
int munmap(
      void addr[length], size_t length);
```

- The prot argument describes the desired memory protection of the mapping
- We need PROT_READ and PROT_WRITE for a v4l2 buffer.
- The flags argument determines whether updates to the mapping are visible to other processes mapping the same region, and whether updates are carried through to the underlying file.
- In v4l2 we use MAP_SHARED by default.

# Querying Each Buffer and MMAP

```
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/videodev2.h>
#define NUM_BUFFERS 4
struct buffer {
    void *start;
    size_t length;
};
static struct buffer buffers[NUM_BUFFERS];
for (i = 0; i < req.count; i++) {
        struct v4l2_buffer buf;
        memset(&buf, 0, sizeof(buf));
        buf.type = V4L2_BUF_TYPE_VIDEO_CAPTURE;
        buf.memory = V4L2_MEMORY_MMAP;
        buf.index = i;
        if (ioctl(fd, VIDIOC_QUERYBUF, &buf) == -1) {
            perror("VIDIOC_QUERYBUF");
            close(fd);
            return EXIT_FAILURE;
        }
        buffers[i].length = buf.length;
        buffers[i].start = mmap(
            NULL, buf.length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd, buf.m.offset
        );
    }
```

- To initialize each buffer memory we will be using ioctl VIDIOC_QUERYBUF and passing the v4l2_buffer structure with its buffer index and type.
- It will give the requested buffer in the structure and we need to mmap it.

# ioctl VIDIOC_QUERYBUF

- This ioctl can be used to query the status of a buffer at any time after buffers have been allocated with the ioctl VIDIOC_REQBUFS ioctl.
- Applications set the type field of a struct v4l2_buffer to the same buffer type as was previously used with struct v4l2_format type and struct v4l2_requestbuffers type, and the index field.
- Valid index numbers range from zero to the number of buffers allocated with ioctl VIDIOC_REQBUFS (struct v4l2_requestbuffers count) minus one.

# struct v4l2_buffer

| Field | Type | Description |
|---|---|---|
| index | __u32 | Identifies the buffer by its index number. |
| type | __u32 | Type of the buffer or stream. |
| bytesused | __u32 | Number of bytes occupied by valid data. |
| flags | __u32 | Buffer status and state flags. |
| field | __u32 | Field order of the video frame. |
| timestamp | struct timeval | Timestamp associated with the buffer. |
| m.offset | __u32 | Offset of the buffer from the start of device memory. |
| length | __u32 | Offset of the buffer from the start of device memory. |

- For the single-planar API, the m.offset contains the offset of the buffer from the start of the device memory, the length field its size.
- The above is a slice of the structure , for full structure refer official documentation.

# Queue of Buffers

## ioctl VIDIOC_QBUF

- Applications call the VIDIOC_QBUF ioctl to enqueue an empty buffer in the driver’s incoming queue.

## ioctl VIDIOC_DQBUF

- Applications call the VIDIOC_DQBUF ioctl to dequeue a filled buffer from the driver’s outgoing queue.

They just set the type, memory and reserved fields of a struct v4l2_buffer, when VIDIOC_QBUF or VIDIOC_DQBUF is called with a pointer to this structure the driver fills the remaining fields or returns an error code.

# Streaming

## ioctl VIDIOC_STREAMON

- The VIDIOC_STREAMON and VIDIOC_STREAMOFF ioctl start and stop the capture or output process.
- Capture hardware is disabled and no input buffers are filled (if there are any empty buffers in the incoming queue) until VIDIOC_STREAMON has been called.
- The ioctl will succeed when at least one output buffer is in the incoming queue.
- If VIDIOC_STREAMON fails then any already queued buffers will remain queued.

## ioctl VIDIOC_STREAMOFF

- The VIDIOC_STREAMOFF ioctl removes all buffers from the incoming and outgoing queues. That means all images captured but not dequeued yet will be lost.
- I/O returns to the same state as after calling ioctl VIDIOC_REQBUFS and can be restarted accordingly.

# Analogy




**Going to a Shop → open("/dev/video0")**

# Analogy




**Identifying the Shop → VIDIOC_QUERYCAP**

# Analogy




**Seeing the Wall Catalog → VIDIOC_ENUM_FMT**

# Analogy




“Ask for 4 Empty Cans → VIDIOC_REQBUFS

# Analogy




**“Read Buffer Information → VIDIOC_QUERYBUF”**

# Analogy




**“Give a Can to the Shopkeeper → VIDIOC_QBUF”**

# Analogy




**“Start Subscription → VIDIOC_STREAMON”**

# Analogy




**Ask for Filled Can → VIDIOC_DQBUF**

# Analogy




“ QBUF -> DBUF -> QBUF ..... Loop

# Analogy




**Stop Subscription → VIDIOC_STREAMOFF**

# Multi-Planar Format

- A video frame can be divided into multiple memory planes, where each plane stores a different part of the image data.
- Instead of treating the frame as one block of memory, V4L2 can describe it as multiple planes.




Single Planar
Memory Layout




Multi Planar (Eg : NV12)
Memory Layout

# Single Vs Multi Planar

| Single Planar | Multi Planar |
|---|---|
| One plane | Multiple planes |
| V4L2_BUF_TYPE_VIDEO_CAPTURE | V4L2_BUF_TYPE_VIDEO_CAPTURE_MPLANE |
| fmt.pix | fmt.pix_mp |
| Buf.m.offset | Buf.m.planes[p].m.mem_offset |

- Creating Buffer

```
struct v4l2_plane planes[VIDEO_MAX_PLANES];
struct v4l2_buffer buf = {
    .type = V4L2_BUF_TYPE_VIDEO_CAPTURE_MPLANE,
    .m.planes = planes,
    .length = 2,
};
```

- MMAP

```
buffers[i].planes[p].length =
        buf.m.planes[p].length;
    buffers[i].planes[p].start =
        mmap(NULL, buf.m.planes[p].length,
            PROT_READ | PROT_WRITE,
            MAP_SHARED,
            fd, buf.m.planes[p].m.mem_offset);
```

# MMAP vs USERPTR Methods

## MMAP

- Driver allocates the video buffers.
- Application calls VIDIOC_REQBUFS.
- Application calls VIDIOC_QUERYBUF to get buffer offset and size.
- Application calls mmap() to access the driver-allocated memory.
- Buffer is then passed using VIDIOC_QBUF.

## USERPTR

- Application allocates the memory.
- Application calls VIDIOC_REQBUFS to initialize the buffer queue.
- VIDIOC_QUERYBUF is not needed to obtain the user memory.
- Application passes the memory address using VIDIOC_QBUF.
- Driver uses/prepares that user-provided memory for the capture operation.

# V4L2 Controls

- Controls are used to configure and query device parameters such as: Brightness, Contrast, Saturation, Exposure, Gain, White balance
- Each control is identified by a Control ID.
- ioctl VIDIOC_QUERYMENU is used to list the supported controls
- ioctl VIDIOC_QUERYCTRL is used to get maximum, minumum and default value of a control.
- ioctl VIDIOC_S_CTRL is used to set a control value.
- ioctl VIDIOC_G_CTRL is used to get a control current value.

# V4L2 Controls

```
int fd = open("/dev/video0", O_RDWR);

    struct v4l2_queryctrl q = {
        .id = V4L2_CID_BRIGHTNESS
    };

    ioctl(fd, VIDIOC_QUERYCTRL, &q);

    struct v4l2_control c = {
        .id = V4L2_CID_BRIGHTNESS
    };

    ioctl(fd, VIDIOC_G_CTRL, &c);
```

```
printf("Brightness: current=%d min=
%d max=%d default=%d\n",
            c.value, q.minimum,
q.maximum, q.default_value);

    printf("Enter new value: ");
    scanf("%d", &c.value);

    ioctl(fd, VIDIOC_S_CTRL, &c);

    close(fd);
```

# References

- https://www.kernel.org/doc/html/v4.12/media/uapi/v4l/v4l2.html
- https://en.wikipedia.org/wiki/Video4Linux
- https://man7.org/linux/man-pages/index.html